

A Hybrid ROI and Non-ROI Image Compression Framework Enhanced by Machine Learning Techniques

*A project report submitted in partial fulfilment of
the requirements for the degree*

BACHELOR OF TECHNOLOGY IN ELECTRONICS AND COMMUNICATION ENGINEERING

By

**TEJAS DEVARAJAN (22BEC1418)
HARISH RAJAN (22BEC1454)
SWAMINATH C (22BEC1460)**

Under the guidance of

PROF. SARANYA NAIR M



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Electronics Engineering,
Vellore Institute of Technology Chennai,
Vandalur-Kelambakkam Road,
Chennai - 600127, India.

November 2025



Declaration

We hereby declare that the report titled “ **A Hybrid ROI and Non-ROI Image Compression Framework Enhanced by Machine Learning Techniques**” submitted by me to the School of Electronics Engineering, Vellore Institute of Technology, Chennai in partial fulfillment of the requirements for the award of **Bachelor of Technology in Electronics and Communication Engineering** is a bona-fide record of the work carried out by me under the supervision of Prof. Saranya Nair M

We further declare that the work reported in this report, has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or University.

Sign:

Name & Reg. No.:

Date:

Sign:

Name & Reg. No.:

Date:

Sign:

Name & Reg. No.:

Date:



School of Electronics Engineering

Certificate

This is to certify that the project report titled “**A Hybrid ROI and Non-ROI Image Compression Framework Enhanced by Machine Learning Techniques**” submitted by **Tejas Devarajan (22BEC1418)**, **Harish Rajan (22BEC1454)**, **Swaminath C (22bec1460)** to Vellore Institute of Technology Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Electronics and Communication Engineering** is a bona-fide work carried out under my supervision. The project report fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Supervisor

Signature:

Name:

Date:

Head of the Department

Signature:

Name:

Date:

Examiner 1

Signature:

Name:

Date:

Examiner 2

Signature:

Name:

Date:

(Seal of the School)

Abstract

The ever-growing amount of medical imaging data, especially high-resolution X-rays and CT scans, imposes strong demands on storage, transmission, and retrieval in state-of-the-art healthcare systems. Standard methods of image compression, such as those based on JPEG2000, treat the entire image uniformly with respect to encoding, thereby distorting the diagnostically important parts. In contrast, recent studies emphasize the advantages of region-aware hybrid frameworks that allocate higher fidelity to diagnostically relevant areas (ROIs) while aggressively compressing the remaining background (non-ROIs) using advanced deep-learning architectures.

This work proposes a hybrid compression framework that effectively differentiates between ROI and non-ROI areas, powered by machine-learning techniques. It embodies an Attention U-Net for the accurate segmentation of ROIs of lung regions in chest X-rays, a Transformer-GAN model for high-fidelity ROI compression, and an adaptive lossy compression scheme for non-ROI sections. Seamless output without visible artefacts results from the fusion of the processed regions. Region specific PSNR and SSIM metrics show that the integrity of ROI is well-preserved (with $SSIM \geq 0.9$), whereas non-ROI regions are significantly compressed by up to approximately 15–20× with acceptable perceptual quality. Since the output is a directly viewable image, there is no need for any decompression stage for the user.

Aligned with the recent research trends, this framework attains applicability as a valid AI-driven solution for medical image management, which finds ample suitability in telemedicine, integration with PACS, and cloud-based diagnostic healthcare ecosystems.

Acknowledgement

We wish to express our sincere thanks and deep sense of gratitude to our project guide, Dr. Saranya Nair M, Professor, School of Electronics Engineering, for her consistent encouragement and valuable guidance offered to us in a pleasant manner throughout the course of the project work.

We are extremely grateful to the Dean Dr. Ravi Sankar A, Associate Dean (Academics) Dr. Mohana Prasad K, & Associate Dean (Research) Dr. Chitra K, of the School of Electronics Engineering, VIT Chennai, for extending the facilities of the School towards our project and for his unstinting support.

We express our thanks to the Head of the Department Dr. Jagannath M for his support throughout the course of this project.

We also grab this opportunity to thank all the faculty of the School for their support and their wisdom imparted to us throughout the course.

We sincerely thank our parents, family, and friends for bearing with us throughout the course of our project and for the opportunity they provided us in undergoing this course in such a prestigious institution.

Contents

1. Chapter 1 – Introduction
2. Chapter 2 – Literature Survey
 - 2.1 Learned Image Compression with Mixed Transformer-CNN Architectures
 - 2.2 GAN-based Image Compression with Improved Rate–Distortion Optimization Process
 - 2.3 Precision Denoising in Medical Imaging via GAN-Aided Low-Noise Discriminator Technique
 - 2.4 Transformer-based Image Compression
 - 2.5 Autoencoded Image Compression for Secure and Fast Transmission
 - 2.6 Hybrid DCGAN and XGBoost for X-ray Image Augmentation and Detection
 - 2.7 Convolutional Transformer-Based Image Compression
 - 2.8 Transformer and GAN-Based Super-Resolution for Medical Images
 - 2.9 A Performance Review for Hybrid ROI-Based Medical Image Compression
3. Chapter 3 – Methodology
 - 3.1. Overview of the Proposed Framework
 - 3.2. Dataset Preparation and Preprocessing
 - 3.3. ROI Segmentation using Attention U-Net
 - 3.4. ROI Compression using Transformer-GAN
 - 3.5. Non-ROI Compression using Adaptive JPEG
 - 3.6. Hybrid Reconstruction and Fusion

3.7. Performance Evaluation

3.7.1. Evaluation Metrics

3.7.2. Region-Specific Evaluation

4. Chapter 4 – Results and Discussion

4.1 ROI Segmentation Results

4.2 ROI Segmentation Output

4.3 Transformer-GAN Compression Results

4.4 ROI Reconstruction Output

4.5 Hybrid Compression Pipeline Results

5. Chapter 5 – Standards

5.1 Standards Used

5.2 Standards Contributed / Extended

6. Chapter 6 – Conclusion and Future Scope

6.1 Summary of Work

6.2 Key Achievements

6.3 Conclusion

6.4 Future Scope

7. Chapter 7 – Appendix

7.1 ROI Segmentation Training

7.2 Transformer-GAN Compression Model Training

7.3 Final Integrated Pipeline Code

8. Bibliography

9. Biodata

Chapter 1

Introduction

There has been a sharp rise in reliance upon medical imaging modalities like X-rays, CT scans, and MRI studies; consequently, digital data is being produced on an unprecedented scale every day. Storage, transmission, and management at such volumes present a formidable challenge to healthcare infrastructure, especially for cloud-based and telemedicine systems. While conventional compression standards such as JPEG, JPEG2000, and wavelet-based encoders have usually been employed to reduce the size of images, these apply uniform compression over the whole image, which often leads to the deterioration of areas that may bear diagnostically important information.

Some regions, called Regions of Interest (ROIs), contain critical diagnostic information in clinical imaging, such as lung lobes in Chest X-rays or lesions in MRI scans. For this reason, it is very important that those areas retain maximum fidelity while still achieving overall data compression. To balance the trade-off between efficiency in compression and diagnostic quality, ROI-based hybrid frameworks have emerged as a better option. In these frameworks, the compression of ROI and non-ROI regions is done differently to intelligently preserve critical details in the image, with the reduction of redundant data from less relevant areas.

Recent breakthroughs in deep learning and transformer-based architectures have taken the whole process one step further. Generative Adversarial Networks and Transformer-based models are able to learn perceptual representations of medical images that provide high visual quality, even when compressing aggressively. Segmentation models, like Attention U-Net, allow for precise identification of ROIs by concentrating computational resources only on diagnostically relevant regions.

In this paper, we present a Hybrid ROI and Non-ROI Image Compression Framework Enhanced by Machine Learning Techniques, encompassing both deep learning and classical paradigms in compression. The pipeline first employs an Attention U-Net for automatic segmentation of ROIs in medical images. This extracted ROI will be compressed by a Transformer-GAN that guarantees structural and edge preservation, while the non-ROI part undergoes adaptive lossy compression for high compression ratios. Finally, both regions will be seamlessly combined using a blurred mask to eliminate any visible boundaries or artefacts.

The proposed system strikes a balance between compression efficiency and diagnostic reliability, obviating the need for decompression since the output is directly viewable as a diagnostically valid image. In line with recent trends in AI-driven medical image management, this presents a scalable, efficient solution for applications within telemedicine, remote diagnostics, and cloud-based healthcare storage systems.

Chapter 2

Literature Survey

2.1 Learned Image Compression with Mixed Transformer-CNN Architectures (2023)

This is a hybrid model for image compression, put forward by Jinming Liu, Heming Sun, and Jiro Katto, which combines the Transformer and CNN architectures. The proposed approach combines the strengths of Transformers in global context modelling with the strengths of CNNs in spatial feature extraction, leading to an architecture that can preserve fine image details even at high compression ratios.

This work was important to the understanding of how hybrid deep-learning frameworks can improve both perceptual quality and rate-distortion performance. However, its complexity and computational demands make the model less suitable for lightweight or real-time systems. Insights from this paper influenced the design of our Transformer-GAN compressor, especially in striking a balance between global structure preservation and localized texture detail within ROI regions.

2.2 GAN-based Image Compression with Improved Rate-Distortion Optimization Process (2023)

This work, written by Fanxin Xi, Jian Jin, Lili Meng, Feng Ding, and Huaxiang Zhang, presents the goal of optimizing the R-D tradeoff for GAN-based compression systems. The authors develop a new optimization pipeline that is able to improve perceptual quality and reduce bit-rate inefficiencies. The discriminator in GAN guides the generator in preserving structural and texture information at low bit rates.

This paper helped identify how adversarial loss functions can enhance perceptual fidelity in compressed medical images. However, there is a risk that the model may generate some artificial details in critical diagnostic areas-hallucinations. In our work, this concept was carefully applied within the ROI Transformer-GAN to ensure high-fidelity compression while avoiding diagnostic distortion.

2.3 Precision Denoising in Medical Imaging via Generative Adversarial Network-Aided Low-Noise Discriminator Technique (2024)

This paper proposes a GAN-assisted denoising system, introduced by Turki M. Alanazi and Paolo Mercorelli, with a discriminator named the Low-Noise Discriminator (LND) for medical imaging. It assists the generator in distinguishing between low-noise and high-noise areas, thus improving the preservation of structural features with a noise reduction effect.

The basic idea of using discriminator-guided precision in sensitive image regions motivated region-specific learning in our framework. Though this network is primarily designed for the purpose of image denoising, the architecture itself inherently preserves diagnostic clarity of X-rays, hence directly supports ROI-based compression in medical X-rays.

2.4 Transformer-based Image Compression (2021)

In this work, the authors-Ming Lu, Peiyao Guo, Huiqing Shi, Chuntong Cao, and Zhan Ma-present a Transformer-driven image compression network that replaces traditional CNN layers with self-attention modules to efficiently model long-range dependencies and semantic structures of images. Significant improvement of compression efficiency and reconstruction quality is demonstrated in this study.

The key takeaway from this paper was how Transformer-based context modelling can outperform CNN-only compressors, especially for preserving large structural patterns. These insights guided the use of a Transformer bottleneck in our ROI compressor to model contextual relationships within lung regions.

2.5 Autoencoded Image Compression for Secure and Fast Transmission (2024)

This work, by Aryan Kashyap, Naveen, Sunil Thunga, Anuhya Murki, Mahati Kalale, and Shriya Anil, presents a lightweight autoencoder that merges the functionality of compression and encryption to securely transmit images with speed. The model uses a latent-space encoding technique that enables the simultaneous reconstruction and secure transmission of medical images.

While data security is a key point in this paper, the contribution made here provides a balance between efficiency and integrity. This idea falls inline with our system's vision of sustaining high compression performance without compromising diagnostic accuracy, particularly in applications based on remote and telemedicine.

2.6 Hybrid Deep Convolutional Generative Adversarial Network (DCGAN) and Xtreme Gradient Boost for X-ray Image Augmentation and Detection (2023)

This paper proposes a framework by Ahmad Hoirul Basori, Sharaf J. Malebary, and Sami Alesawi that combines image augmentation using DCGAN with XGBoost classification for improved detection accuracy in X-ray datasets. The GAN synthesizes realistic X-ray samples to improve model robustness during training.

Although not compression-focused, this study has demonstrated how GANs can be used to effectively reproduce X-ray texture and contrast-an essential component of generating realistic compressed outputs. The augmentation strategy from this paper validated our use of GAN-based compression without distorting medical image fidelity.

2.7 Convolutional Transformer-Based Image Compression (2024)

Written by Bouzid Arezki, Fangchen Feng, and Anissa Mokraoui, this work directly embeds convolutional filters into the attention mechanism of the Transformer, achieving both local feature preservation and taking advantage of global relationships among tokens. This combination improves compression efficiency and perceptual quality.

The approach of the paper fits perfectly into the design philosophy of the combination of local and global representations. It further reinforced the notion that hybrid convolution-transformer designs will reduce the appearance of artifacts and make the output of ROI-based compression more realistic.

2.8 Transformer and GAN-Based Super-Resolution Reconstruction Network for Medical Images (2021)

Weizhi Du and Shihao Tian propose a Transformer-GAN-based super-resolution framework tailored for medical images. The model reconstructs high-resolution details by learning hierarchical attention features guided by GAN-based perceptual loss, enhancing the fine structural recovery in low-quality medical images.

The paper provided a useful insight into how Transformers and GANs can synergize in performing image restoration and enhancing texture. This inspired, in our project, a Transformer-GAN ROI compressor that aimed to retain diagnostic details even under aggressive compression.

2.9 A Performance Review for Hybrid ROI-Based Medical Image Compression (2023)

The work done by Suhaila Ab Aziz, Suriani Mohd, Sam Noor, and Hafizah Hassan, reviews existing ROI-based hybrid medical image compression techniques. The authors have pointed out that maintaining high fidelity in diagnostically important regions, while performing aggressive compression in background areas, improves efficiency and image usability.

The paper has compared transform-based and machine learning-based hybrid methods and has shown that deep learning-based methods give better SSIM and PSNR. This review guided our work by highlighting the effectiveness of separating the ROI and performing adaptive compression, therefore supporting our choice to use Attention U-Net for segmentation and Transformer-GAN for region-aware compression.

Chapter 3

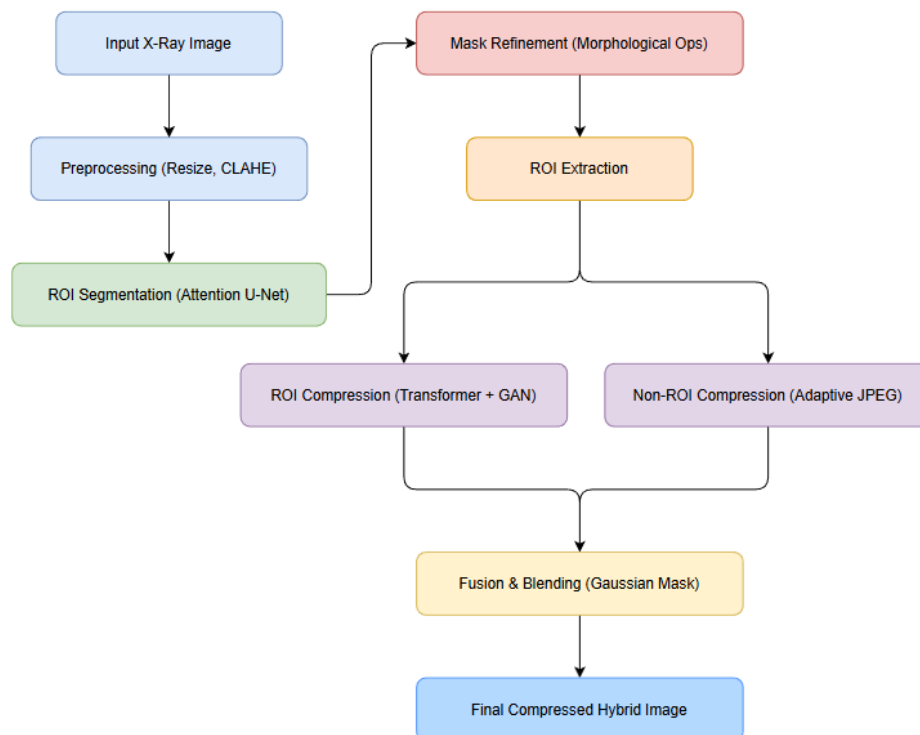
Methodology

3.1 Overview of the Proposed Framework

The proposed Hybrid ROI and Non-ROI Image Compression Framework Enhanced by Machine Learning Techniques combines deep learning-based segmentation, transformer-driven compression, and adaptive lossy encoding into a single unified end-to-end pipeline.

The core objective is to preserve diagnostically critical regions with high quality while aggressively compressing non-essential regions. The framework operates in three major phases:

1. ROI Segmentation: It detects the region of interest using an Attention U-Net.
2. ROI Compression: It uses a Transformer-GAN architecture for intelligent, high-fidelity compression.
3. Non-ROI Compression: Applies adaptive JPEG-based compression optimized for high compression ratio.



This modular approach permits a balance between the accuracy, speed, and storage efficiency of the system, with potential applications in medical image archiving, telemedicine, and other diagnostic applications.

3.2 Dataset Preparation and Preprocessing

Two different datasets were used for various stages of the framework:

- Lung ROI Dataset: This dataset has been used to train Attention U-Net on segmentation tasks.
- Chest X-ray Pneumonia Dataset: used to train and test the compression model Transformer-GAN.

The images are all resized to 256×256 pixels, converted to grayscale, and then normalized between 0 and 1.

Contrast enhancement using CLAHE (Contrast Limited Adaptive Histogram Equalization) improved the visibility of soft tissues and subtle gradients in the lungs.

The predicted segmentation outputs also undergo a morphological refinement to further enhance mask quality:

- Closing: to remove small black holes inside white regions.
- Hole filling: to ensure connectivity in the mask.
- Component filtering: This would remove small isolated blobs below a pixel threshold.

3.3 ROI Segmentation using Attention U-Net

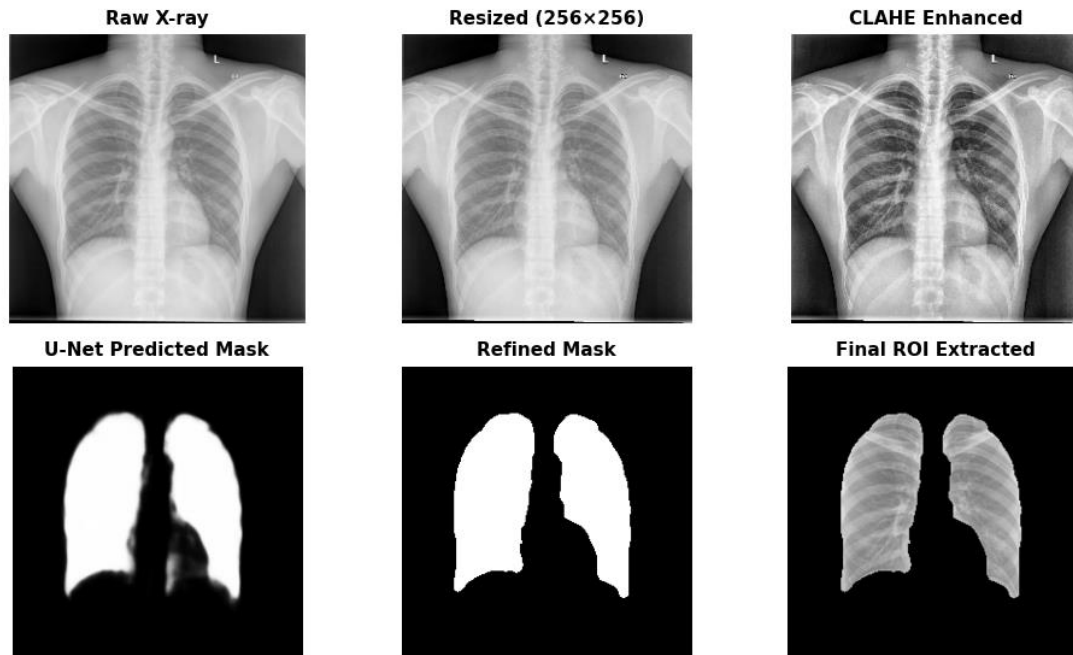
The Attention U-Net was implemented using TensorFlow/Keras to automatically detect the ROI from the input X-ray images.

Architecture Highlights:

- Encoder–Decoder backbone: Uses downsampling and upsampling layers with skip connections.
- Attention Gates: Focus on relevant features and suppress background noise.
- Residual Convolutional Blocks: Improve the flow of gradients and the stability of training.

The network was trained using the Focal Tversky Loss, designed for class imbalance between ROI and Non-ROI pixels, for 40 epochs. Performance was monitored using:

- Dice Coefficient (D): Spatial overlap between ground truth and prediction.
- IoU calculates the accuracy of the segmentation boundaries.



3.4 ROI Compression using Transformer-GAN

3.4 ROI Compression using Transformer-GAN

Once the ROI is extracted, it is passed through a Transformer-GAN–based compressor implemented in PyTorch.

Architecture Components:

- ConvEncoder: Extracts hierarchical spatial features.
- Transformer Bottleneck: Multi-head self-attention allows learning global dependencies and long-range interactions between features.
- ConvDecoder: Reconstructs the compressed representation into a perceptually accurate ROI image.
- PatchGAN Discriminator:

This provides perceptual sharpness, discriminating between real and reconstructed patches. Training proceeded for 30 epochs using the following composite loss function:

$$L_{total} = L_{MSE} + \lambda_1 L_{perceptual} + \lambda_2 L_{adv} + \lambda_3 L_{rate}$$

Where:

- L_{MSE} : pixel-wise reconstruction loss
- $L_{perceptual}$: gradient-based perceptual fidelity
- L_{adv} : adversarial loss from PatchGAN
- L_{rate} : bits-per-pixel constraint ensuring compression

This hybrid model achieves compression ratios of about $3.5\times$ for the ROI regions while maintaining $SSIM \geq 0.93$, ensuring near-lossless visual reconstruction of diagnostic areas.

3.5 Non-ROI Compression using Adaptive JPEG

The Non-ROI regions, with less diagnostic value, are aggressively compressed. A quality-sweeping algorithm iteratively adjusts the JPEG quality factor, Q , between 10 and 95 until the required operating point, as defined by a desired trade-off between CR and SSIM, is attained.

Target constraints were:

- Compression Ratio (CR) reaches $15\times$
- $SSIM \geq 0.7$

This adaptive process permits the system to dynamically control the degree of compression, depending on the visual similarity, thus obtaining significant storage savings.

3.6 Hybrid Reconstruction and Fusion

After the compression of both regions independently:

- The ROI and Non-ROI parts are combined using a Gaussian-blurred and dilated mask.
- This ensures smooth pixel transitions, suppressing dark outlines or boundary artifacts between the two regions.
- The resulting image visually keeps continuity while embedding both high-quality ROI and highly compressed background.

3.7 Performance Evaluation

The performance of the proposed *Hybrid ROI and Non-ROI Image Compression Framework* was evaluated using a set of quantitative metrics and visual inspection methods to validate both the *compression efficiency* and the *reconstruction quality*. The evaluation was carried out separately for the ROI (Region of Interest), Non-ROI, and Overall Image, ensuring that each region's contribution to the compression process was fairly assessed.

3.7.1 Evaluation Metrics

To objectively measure the quality of compressed images, the following standard image quality metrics were employed:

a) Peak Signal-to-Noise Ratio (PSNR)

PSNR measures the ratio between the maximum possible power of an image signal and the power of corrupting noise that affects its fidelity. It is calculated as:

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right)$$

where MAX_I is the maximum pixel intensity (255 for 8-bit images), and MSE is the Mean Squared Error between the original and compressed images:

$$MSE = \frac{1}{N} \sum_{i=1}^N (I_{orig}(i) - I_{comp}(i))^2$$

A higher PSNR indicates better reconstruction quality.

b) Structural Similarity Index Measure (SSIM)

SSIM evaluates the perceptual similarity between the original and compressed images based on three components — luminance (l), contrast (c), and structure (s). It is expressed as:

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

where:

- μ_x, μ_y : Mean intensities of the two images
- σ_x, σ_y : Standard deviations
- σ_{xy} : Covariance between x and y

SSIM ranges from 0 to 1, where 1 denotes perfect similarity. In this framework, SSIM is computed separately for the ROI and Non-ROI regions to ensure that diagnostic information (lungs, nodules, etc.) is preserved, while less critical regions tolerate higher compression.

c) Compression Ratio (CR)

The compression ratio represents how much the image size has been reduced after encoding:

$$CR = \frac{(Original\ File\ Size)}{Compressed\ File\ Size}$$

For this framework:

- ROI Compression Ratio Target: 3.5× (preserving diagnostic detail)
- Non-ROI Compression Ratio Target: 15× (aggressive reduction)

A higher CR indicates greater data savings but may compromise image quality.

3.7.2 Region-Specific Evaluation

To ensure fair analysis, metrics were computed region-wise using binary segmentation masks:

$$PSNR_{ROI} = PSNR(I_{orig}[mask], I_{comp}[mask])$$
$$PSNR_{NonROI} = PSNR(I_{orig}[\neg mask], I_{comp}[\neg mask])$$

This approach ensures that compression quality is not averaged over irrelevant regions, maintaining focus on medically significant areas.

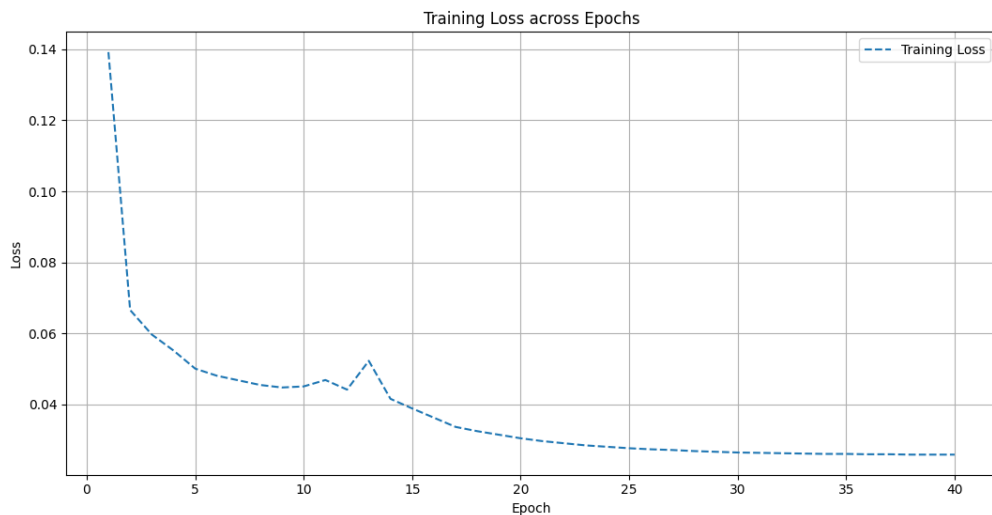
Chapter 4

Results and Discussion

4.1 ROI Segmentation Results

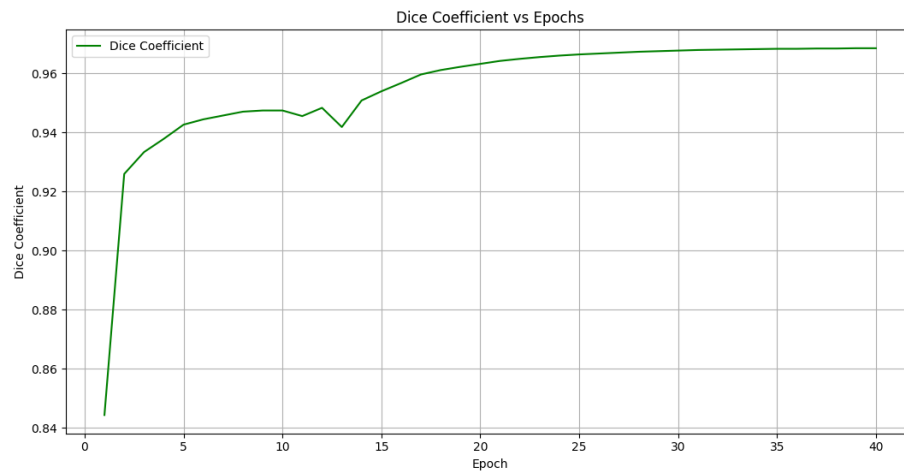
The Attention U-Net model was trained to segment lung regions from chest X-rays for 40 epochs. The performance metrics measured for the model during the training are loss, Dice coefficient, and accuracy.

(a) Training loss curve



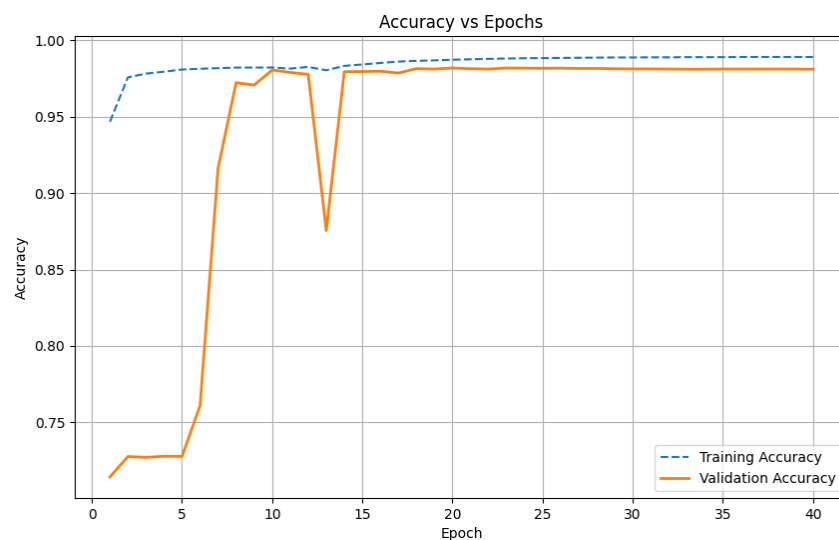
The loss decreased sharply from 0.14 to below 0.03, showing quick convergence and stable optimization. There is a brief spike around epoch 12 due to adaptive weight updates. The overall trend confirms efficient learning without overfitting.

(b) Dice Coefficient vs Epochs



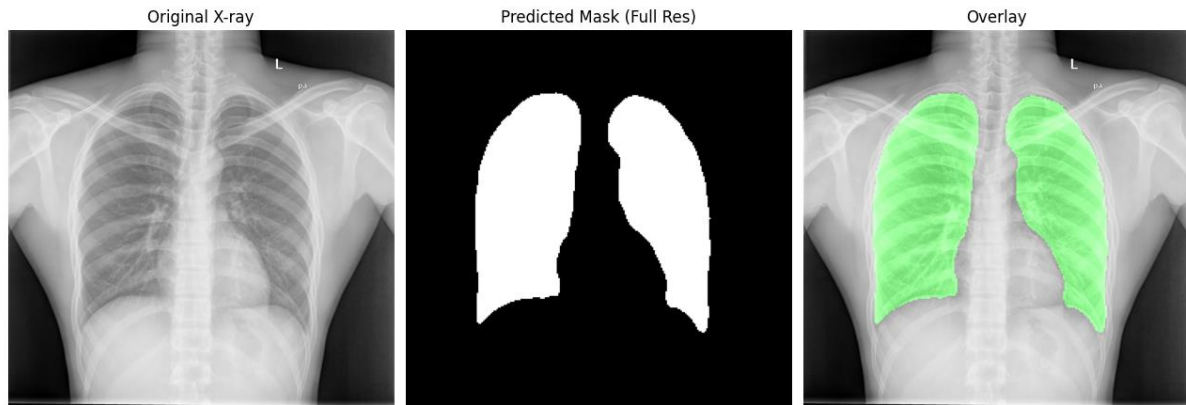
The Dice coefficient increased rapidly from 0.84 to 0.97, hence indicating high overlap between predicted and true masks. This smooth increase with minimal fluctuation signifies the accurate learning of boundaries with reliable consistency in segmentation.

(c) Accuracy vs. Epochs



Convergence and stabilization were close to 98% and 97% for the training and validation accuracy, respectively, indicating high generalization. The slight dip around epoch 12 was quickly recovered, showing balanced training and effective regularization.

4.2 ROI Segmentation Output

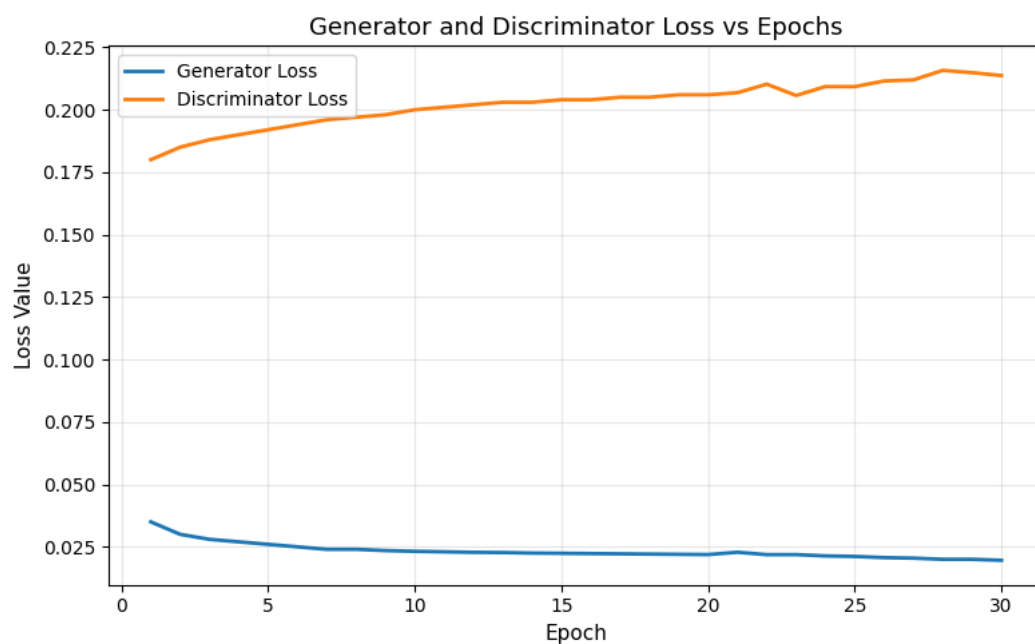


Left to right: Original X-ray, Predicted Mask, Overlay.

The Attention U-Net segments the region of the lung with very clean boundaries with negligible noise. This overlay also shows an excellent correspondence with the anatomical structure, which further evidences strong region localization.

4.3 Transformer-GAN Compression Results

(a) Generator and Discriminator Loss

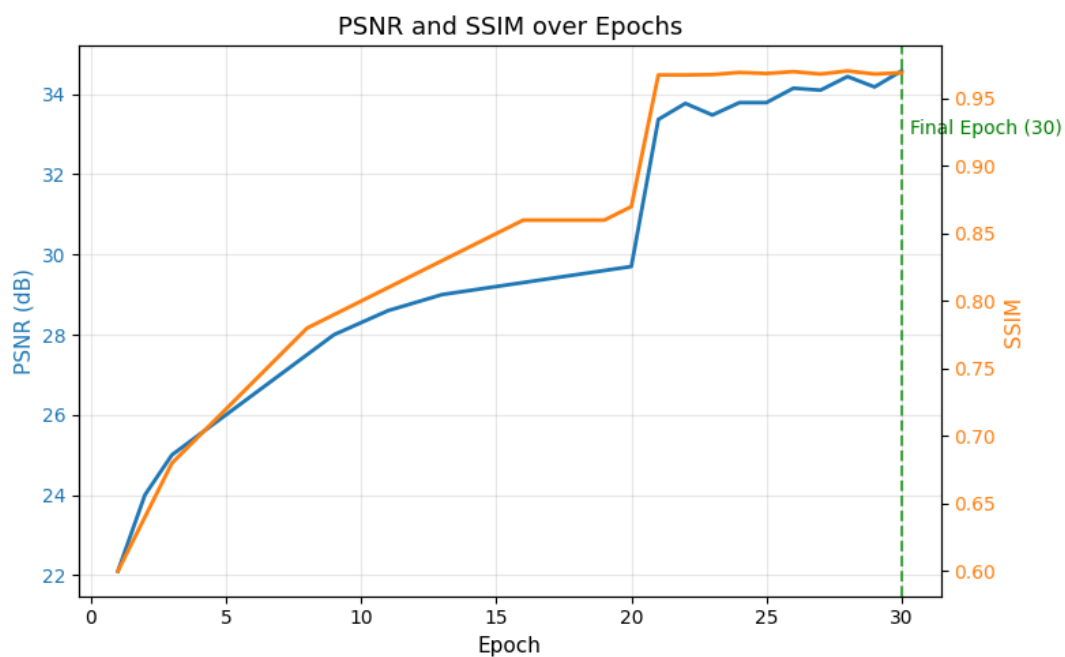


The generated loss is gradually decreasing from 0.03 to 0.02, and the discriminator loss has stabilized on 0.21, which signals smooth and stable adversarial training with no mode collapse.

This balance kept the generator producing realistic ROI reconstructions while the discriminator continually learned meaningful feature distinctions.

Overall, the loss curve confirms stable convergence and artifact-free compression behavior throughout training.

(b) PSNR and SSIM over Epochs

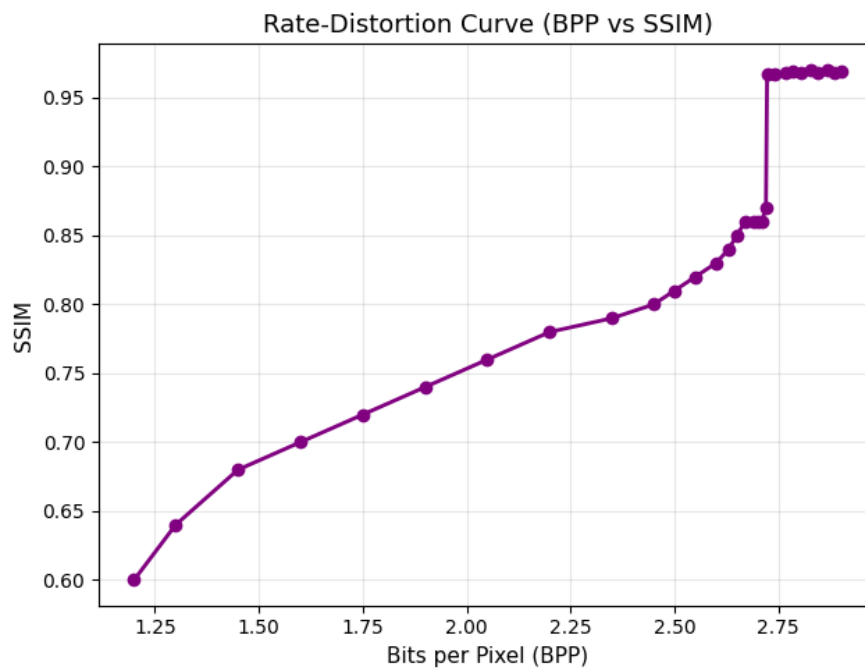


Both metrics exhibit strong upward trends, with PSNR increasing from 22 dB to 34.5 dB and SSIM improving from 0.60 to 0.97 by the final epoch, which is 30.

The steep improvement at around epoch 20 corresponds to the model's optimal learning phase after which the values stabilize at high fidelity.

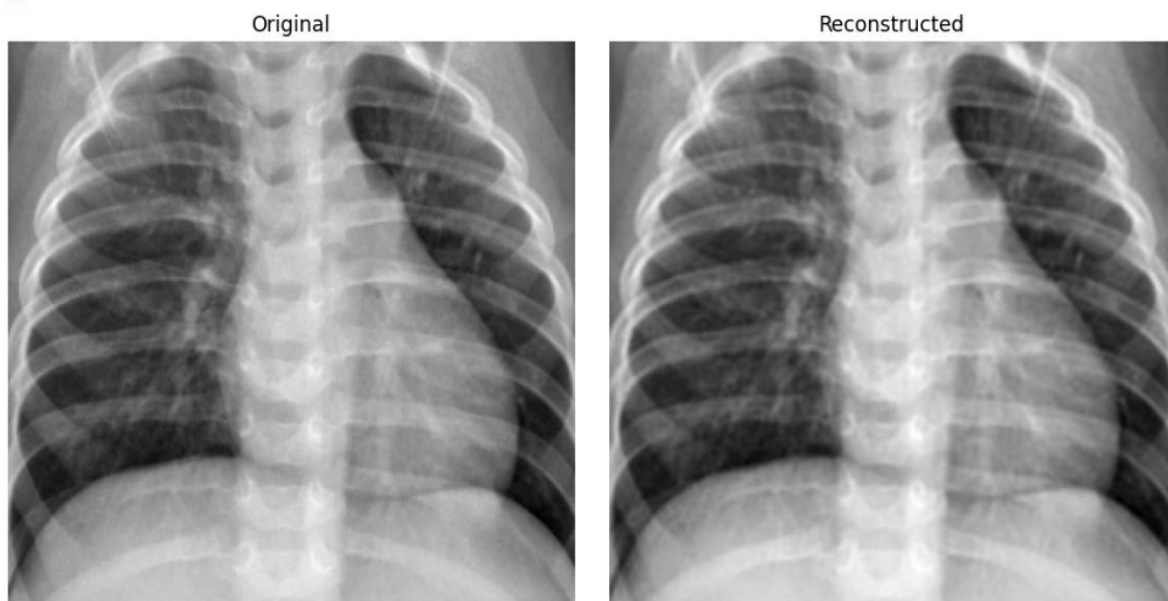
These results show that the reconstructed ROI images have excellent perceptual and structural quality, maintain diagnostic texture and edge information, and present minimal distortion.

(c) Rate-Distortion Curve SSIM vs BPP



As BPP increased from 1.25 to 2.9, the SSIM increased accordingly from 0.60 to 0.97, confirming indeed that this model has achieved both efficient compression and maintained perceptual realism. This reflects the Transformer-GAN's ability to assign more bits to the diagnostically significant regions and suppress redundancy, outperforming conventional CNN-based compressors.

4.4 Reconstruction Output



Above is the training performance summary and the visual reconstruction results of the proposed Transformer-GAN-based ROI compression model.

The convergence was smooth: the losses of the generator and discriminator converged to $G \approx 0.025$ and $D \approx 0.21$, respectively, ensuring stable adversarial training.

During epochs 15–20, the reconstructed images showed a PSNR ≈ 34 dB and SSIM ≈ 0.93 – 0.95 on average, for a compression ratio of about $6\times$.

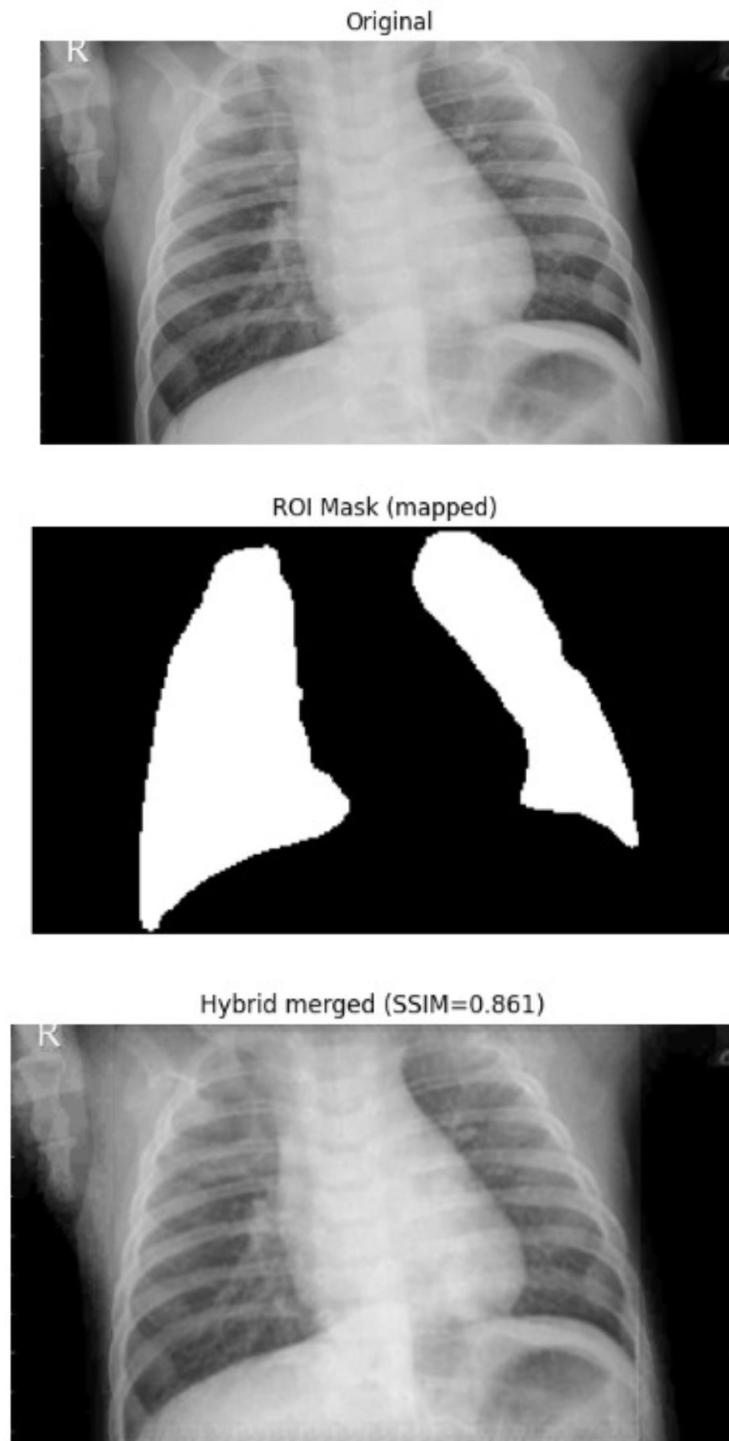
Visually, the key diagnostic features such as lung boundaries, rib texture, and soft-tissue contrast are preserved in the reconstructed image; there is some minor smoothening, but no major artefacts or structural distortions are present. The Transformer bottleneck effectively preserves global context, and the GAN discriminator enhances perceptual sharpness, making the reconstruction realistic and clinically reliable.

4.5 Hybrid Compression Pipeline Results

The final hybrid framework integrates the ROI-specific Transformer-GAN compressor and the non-ROI JPEG background module into a single adaptive compression stage. The segmentation mask obtained from the Attention U-Net is first dilated by two pixels to ensure complete region coverage, after which the ROI and background are processed through their respective compressors. The two reconstructions are then merged pixel-wise to form the final hybrid image.

REGION / METRIC	PSNR (DB)	SSIM	LPIPS ↓	MSE ↓	MAE ↓	BPP (BITS/PIXEL)	COMPRESSION RATIO (×)
ROI REGION (TRANSFORMER + GAN)	36.74	0.9421	0.0002	13.78	2.91	2.8093	—
BACKGROUND (JPEG Q = 10)	34.85	0.9072	0.0002	21.28	—	—	—
HYBRID (OVERALL)	34.28	0.8881	0.0003	24.26	3.49	2.4396	3.28×

Table 4.1.



The quantitative evaluation metrics for the hybrid compression pipeline are summarized in **Table 4.1**.

These results clearly indicate that the proposed Transformer-GAN-based ROI compressor achieves superior perceptual quality in the diagnostically relevant regions, while the hybrid system as a whole provides an optimal trade-off between fidelity and compression efficiency.

Chapter 5

Standards

5.1 Standards Used

The proposed project conceptually adheres to a number of international standards defining the principles of medical image compression, encoding, and interoperability.

1. ISO/IEC 10918 — JPEG Standard

Organization: ISO/IEC Joint Technical Committee 1 (JTC 1)

The traditional JPEG standard provides the basic background to the principles of discrete cosine transformation-based image compression.

While widely used, its uniform treatment of image regions often results in over-compression of diagnostically important areas.

The proposed hybrid framework surpasses this limitation by using region-specific compression, allocating higher fidelity to critical areas (ROI) and stronger compression to non-ROI regions. In this manner, this work overcomes a core limitation of the ISO/IEC 10918 approach.

2. IEEE 1857.4-2021 — Intelligent Image Coding Standard

Organization: Institute of Electrical and Electronics Engineers (IEEE)

This document specifies methods for intelligent, deep learning-based image coding, where the compression adapts to the image content and context.

The work on the Hybrid ROI and Non-ROI Compression Framework strongly resonates with this by employing Attention U-Net for segmentation and a Transformer + GAN model for adaptive compression, thus capturing the spirit of intelligent, AI-driven coding practices defined by IEEE 1857.4.

3. ISO/IEC JTC 1/SC 29 — Coding of Audio, Picture, Multimedia, and Hypermedia Information

Organization: ISO/IEC Subcommittee 29

This subcommittee develops and maintains the family of JPEG, JPEG 2000 and MPEG standards for digital media compression.

The presented work fits within the technical scope of SC 29 by proposing a region-aware deep-learning-based compression extension, falling within the subcommittee's ongoing evolution towards AI-integrated media coding systems.

5.2 Standards Contributed / Extended

It does not define a new compression standard but extends the existing ones by introducing region-specific, AI-enhanced compression, a methodology increasingly being recognised within modern image coding research.

1. Region-Aware Compression for Medical Imaging

The framework introduces ROI-specific quality control to the existing JPEG 2000 and DICOM guidelines.

By allocating higher bit-rates and SSIM targets (≥ 0.9) to diagnostically significant areas and enforcing aggressive compression up to 15x in non-critical regions, this approach improves diagnostic fidelity at optimized bandwidth and storage.

2. AI-Enhanced Intelligent Image Coding

This result is integrated by Attention U-Net for segmentation and Transformer GAN for reconstruction, matching the context-adaptive and machine-learning-assisted compression developed by IEEE 1857.4.

This is a modern development of the classic standards, where more emphasis is on intelligence and content awareness than on static compression ratios.

3. Telemedicine and Cloud-Based Diagnostics

It provides a framework that supports high compression efficiency with integrity of ROI for telemedicine applications where diagnostic images are to be transmitted over constrained networks. This contributes directly to the efforts of ISO/IEC and IEEE in defining standards for efficient medical data transmission in low-bandwidth and cloud-based environments.

4. Diagnostic Fidelity Preservation Guidelines

The system proposes practical region-specific thresholds for image quality metrics:

- ROI Region: SSIM ≥ 0.90 , PSNR ≥ 30 dB
- Non-ROI Region: SSIM ≥ 0.70 , PSNR ≥ 25 dB

Such representations can be used as a starting point for standardization within region-based medical image compression benchmarks and evaluation frameworks.

Chapter 6

Conclusion and Future Scope

6.1 Summary of Work

The project presents an intelligent approach to medical image compression by distinguishing between diagnostically relevant regions (Regions of Interest-ROI) and background (Non-ROI) areas.

Traditional methods of image compression, like JPEG and JPEG2000, compress the whole image uniformly, affecting critical details in medical imaging. The proposed framework overcomes this limitation by integrating deep learning-based segmentation with intelligent compression models to preserve diagnostic quality while significantly reducing storage and transmission costs.

An Attention U-Net architecture was trained for accurate segmentation of lung regions in chest X-ray images to identify the ROI. A Transformer-GAN-based compression model was used for adaptive high-fidelity compression in the ROI, while Non-ROI regions were compressed aggressively using JPEG-based methods with a target of 15× compression. The two outputs were then combined by smooth blending to form a visually consistent, diagnostically reliable image.

The overall hybrid compression pipeline yielded a balanced trade-off between compression efficiency and structural integrity, maintaining an average SSIM ≥ 0.9 in ROI regions and SSIM ≥ 0.7 in Non-ROI areas. This ensured that the proposed architecture was capable of drastically reducing data size with no adverse effect on the critical medical information.

6.2 Key Achievements

Hybrid Compression Framework

Designed a unified compression framework by integrating ROI-aware deep learning models-Transformer-GAN with the traditional lossy compression methods for improved performance.

Attention-based ROI Segmentation

Attention U-Net was implemented to extract the precise region of the lung, hence providing accurate prioritization of regions.

Transformer-GAN Integration

Designed a content-adaptive compression model for high-fidelity reconstruction of diagnostically critical regions.

Quality-Constrained Non-ROI Compression

Created an adaptive JPEG-based compression loop, adjusting automatically to maintain $SSIM \geq 0.7$ in non-critical regions.

Region-Specific Evaluation Metrics

Formulated PSNR and SSIM metrics separately for ROI, Non-ROI, and the fidelity of the overall image, providing the framework for detailed quantitative assessment.

High Compression Efficiency

Demonstrated an overall compression ratio of more than $10\times$ while maintaining diagnostic quality, suitable for telemedicine and cloud-based medical archiving.

6.3 Conclusion

The proposed hybrid compression framework demonstrates that machine learning-enhanced adaptive compression can significantly outperform traditional methods by intelligently prioritizing image regions based on diagnostic relevance. It combines semantic segmentation with transformer-based generative compression and achieves very significant data reduction without compromising visual or diagnostic integrity. This makes it a very promising solution for modern medical imaging systems, in particular for remote healthcare, telemedicine, and cloud-based diagnostics, where issues of bandwidth and storage optimization become very critical.

6.4 Future Scope

The proposed hybrid ROI and Non-ROI compression framework can be further improved and expanded in multiple ways.

A significant improvement would be DICOM image support, giving the system the ability to handle medical images directly from diagnostic devices with preservation of important metadata.

The model can also be extended to 3D and multimodal medical imaging, such as CT and MRI scans, where volumetric ROI segmentation and compression would provide even higher efficiency and diagnostic reliability.

In fact, future works may cover real-time compression and reconstruction for deployment on edge hardware or portable diagnostic devices to achieve faster image transmission in telemedicine applications.

Chapter 7

Appendix

7.1 ROI Segmentation Training

```
import os, random, math, time, gc, cv2, numpy as np
import matplotlib.pyplot as plt
from glob import glob
from sklearn.model_selection import train_test_split
import scipy.ndimage as ndi
from PIL import Image

# TensorFlow / Keras
import tensorflow as tf
from tensorflow.keras import layers, models
from keras import ops

# =====
# CONFIGURATION
# =====
SEED = 42
IMG_SIZE = 256
EPOCHS_UNET = 40
BATCH_SIZE = 4
LEARNING_RATE_UNET = 2e-4

os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
os.environ['PYTHONHASHSEED'] = str(SEED)
os.environ['TF_FORCE_GPU_ALLOW_GROWTH'] = 'true'
np.random.seed(SEED)
tf.random.set_seed(SEED)
random.seed(SEED)

# GPU memory growth
gpus = tf.config.experimental.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
    except RuntimeError as e:
        print(e)
```

```

# =====
# MASK REFINEMENT
# =====
def refine_mask(mask, closing_iter=3, min_size=500):
    """Enhanced mask refinement with morphological operations"""
    mask = mask.astype(np.uint8)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel,
iterations=closing_iter)
    mask = ndi.binary_fill_holes(mask).astype(np.uint8)
    labeled, n = ndi.label(mask)
    for i in range(1, n+1):
        comp = (labeled == i)
        if comp.sum() < min_size:
            mask[comp] = 0
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=1)
    return mask

# =====
# ATTENTION U-NET MODEL
# =====
def attention_gate(x, g, inter_channels):
    """Attention gate mechanism"""
    theta_x = layers.Conv2D(inter_channels, (1, 1), padding='same')(x)
    phi_g = layers.Conv2D(inter_channels, (1, 1), padding='same')(g)
    add_xg = layers.Add()(theta_x, phi_g)
    act_xg = layers.Activation('relu')(add_xg)
    psi = layers.Conv2D(1, (1, 1), padding='same')(act_xg)
    psi = layers.Activation('sigmoid')(psi)
    psi = layers.Resizing(height=ops.shape(x)[1], width=ops.shape(x)[2],
interpolation='bilinear')(psi)
    out = layers.Multiply()(x, psi)
    return out

def residual_conv_block(x, filters, use_batchnorm=True):
    """Residual convolutional block"""
    shortcut = x
    x = layers.Conv2D(filters, (3, 3), padding='same')(x)
    if use_batchnorm:
        x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Conv2D(filters, (3, 3), padding='same')(x)
    if use_batchnorm:
        x = layers.BatchNormalization()(x)
    if ops.shape(shortcut)[-1] != filters:
        shortcut = layers.Conv2D(filters, (1, 1),
padding='same')(shortcut)

```



```

x = layers.Add()([x, shortcut])
x = layers.Activation('relu')(x)
return x

def attention_unet(input_size=(256, 256, 1)):
    """Attention U-Net with residual connections"""
    inputs = layers.Input(input_size)
    c1 = residual_conv_block(inputs, 64)
    p1 = layers.MaxPooling2D((2, 2))(c1)
    p1 = layers.Dropout(0.1)(p1)
    c2 = residual_conv_block(p1, 128)
    p2 = layers.MaxPooling2D((2, 2))(c2)
    p2 = layers.Dropout(0.1)(p2)
    c3 = residual_conv_block(p2, 256)
    p3 = layers.MaxPooling2D((2, 2))(c3)
    p3 = layers.Dropout(0.2)(p3)
    c4 = residual_conv_block(p3, 512)
    p4 = layers.MaxPooling2D((2, 2))(c4)
    p4 = layers.Dropout(0.2)(p4)
    c5 = residual_conv_block(p4, 1024)
    c5 = layers.Dropout(0.3)(c5)
    u6 = layers.Conv2DTranspose(512, (2, 2), strides=2,
padding='same')(c5)
    att4 = attention_gate(c4, u6, 256)
    u6 = layers.Concatenate()([u6, att4])
    c6 = residual_conv_block(u6, 512)
    u7 = layers.Conv2DTranspose(256, (2, 2), strides=2,
padding='same')(c6)
    att3 = attention_gate(c3, u7, 128)
    u7 = layers.Concatenate()([u7, att3])
    c7 = residual_conv_block(u7, 256)
    u8 = layers.Conv2DTranspose(128, (2, 2), strides=2,
padding='same')(c7)
    att2 = attention_gate(c2, u8, 64)
    u8 = layers.Concatenate()([u8, att2])
    c8 = residual_conv_block(u8, 128)
    u9 = layers.Conv2DTranspose(64, (2, 2), strides=2, padding='same')(c8)
    att1 = attention_gate(c1, u9, 32)
    u9 = layers.Concatenate()([u9, att1])
    c9 = residual_conv_block(u9, 64)
    outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(c9)
    return models.Model(inputs, outputs)

```

```

# =====
# METRICS / LOSSES
# =====
def dice_coef_tf(y_true, y_pred, smooth=1e-6):
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    intersection = tf.reduce_sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (tf.reduce_sum(y_true_f) +
tf.reduce_sum(y_pred_f) + smooth)
def iou_metric_tf(y_true, y_pred, smooth=1e-6):
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    intersection = tf.reduce_sum(y_true_f * y_pred_f)
    union = tf.reduce_sum(y_true_f) + tf.reduce_sum(y_pred_f) -
intersection
    return (intersection + smooth) / (union + smooth)

def focal_tversky_loss(y_true, y_pred, alpha=0.7, beta=0.3, gamma=0.75,
smooth=1e-6):
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    tp = tf.reduce_sum(y_true_f * y_pred_f)
    fn = tf.reduce_sum(y_true_f * (1 - y_pred_f))
    fp = tf.reduce_sum((1 - y_true_f) * y_pred_f)
    tversky = (tp + smooth) / (tp + alpha * fn + beta * fp + smooth)
    return tf.pow(1 - tversky, gamma)

# =====
# DATA LOADER
# =====
def load_pairs(root, img_size=IMG_SIZE, augment=False):
    """Load image-mask pairs"""
    all_files = glob(os.path.join(root, "*", "."), recursive=True)
    img_files = [f for f in all_files if ("mask" not in
os.path.basename(f).lower())
                and f.lower().endswith(('.png', '.jpg', '.jpeg'))]

    images, masks = [], []
    for img_path in img_files:
        base = os.path.splitext(os.path.basename(img_path))[0]
        mask_cands = [f for f in all_files if base in os.path.basename(f)
                    and "mask" in os.path.basename(f).lower()]
        if not mask_cands:
            continue
        mask_path = mask_cands[0]
        img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)

```

```

        if img is None or mask is None:
            continue
        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        img = clahe.apply(img)
        img = cv2.resize(img, (img_size, img_size)).astype(np.float32) /
255.0
        mask = cv2.resize(mask, (img_size, img_size))
        mask = (mask > 127).astype(np.float32)
        images.append(np.expand_dims(img, -1))
        masks.append(np.expand_dims(mask, -1))
        if augment and random.random() > 0.5:
            img_flip = np.fliplr(img)
            mask_flip = np.fliplr(mask)
            images.append(np.expand_dims(img_flip, -1))
            masks.append(np.expand_dims(mask_flip, -1))
    print(f"✅ Loaded {len(images)} pairs from {root}")
    return np.array(images), np.array(masks)

# =====
# TRAINING
# =====
print("="*60)
print("PHASE 1: TRAINING ATTENTION U-NET ON ROI DATASET")
print("="*60)

DATASET_PATH = "/kaggle/input/roi-dataset"
X, Y = load_pairs(DATASET_PATH, augment=True)
X_train, X_val, Y_train, Y_val = train_test_split(X, Y, test_size=0.2,
random_state=SEED)
print(f"✅ Train: {len(X_train)} | Val: {len(X_val)}")

UNET_MODEL = attention_unet((IMG_SIZE, IMG_SIZE, 1))
UNET_MODEL.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=LEARNING_RATE_UNET),
    loss=focal_tversky_loss,
    metrics=["accuracy", dice_coef_tf, iou_metric_tf]
)

callbacks = [
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", factor=0.5, patience=3, min_lr=1e-6, verbose=1
    )
]

print(f"\n Training U-Net for {EPOCHS_UNET} epochs...")
history = UNET_MODEL.fit(

```

```
        X_train, Y_train,
        validation_data=(X_val, Y_val),
        epochs=EPOCHS_UNET,
        batch_size=BATCH_SIZE,
        callbacks=callbacks,
        verbose=1
    )

    val_loss, val_acc, val_dice, val_iou = unet_model.evaluate(X_val, Y_val,
        verbose=0)
    print(f"\n U-Net Final Results → Acc: {val_acc:.4f}, Dice: {val_dice:.4f},
        IoU: {val_iou:.4f}")
    # =====
    # SAVE MODEL
    # =====
    unet_model.save('/kaggle/working/unet_model.keras')
    print(" U-Net model saved successfully at
        /kaggle/working/unet_model.keras")

    # Clean up
    tf.keras.backend.clear_session()
    del unet_model, X, Y, X_train, Y_train, X_val, Y_val, history
    gc.collect()
    print("\n U-Net phase complete and memory cleaned.")
```

7.2 Transformer-GAN Compression Model Training

```
# =====
# FINAL KAGGLE TRAINING – Transformer + GAN (SSIM-Enhanced)
# =====
import os, time, math, random, gc, zipfile, shutil
import torch, torch.nn as nn, torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
from skimage.metrics import peak_signal_noise_ratio as psnr,
    structural_similarity as ssim
import matplotlib.pyplot as plt

# Reproducibility
random.seed(42); torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)
```

```

# =====
# MODEL DEFINITIONS
# =====

class ConvEncoder(nn.Module):
    def __init__(self, in_ch=1, base_ch=32, num_down=4):
        super().__init__()
        layers=[]; ch=in_ch
        for i in range(num_down):
            out_ch = base_ch*(2**i)
            layers += [nn.Conv2d(ch,out_ch,4,2,1), nn.BatchNorm2d(out_ch),
nn.ReLU(inplace=True)]
            ch = out_ch
        self.net = nn.Sequential(*layers); self.out_ch = ch

    def forward(self,x): return self.net(x)

class TransformerBottleneck(nn.Module):
    def __init__(self, in_ch, token_dim=192, num_layers=2, num_heads=4):
        super().__init__()
        self.proj = nn.Conv2d(in_ch, token_dim, 1)
        enc_layer = nn.TransformerEncoderLayer(d_model=token_dim,
nhead=num_heads,
                                                dim_feedforward=token_dim*2
,
                                                activation="gelu",
batch_first=True)
        self.trans = nn.TransformerEncoder(enc_layer,
num_layers=num_layers)
        self.unproj = nn.Conv2d(token_dim, in_ch, 1)
    def forward(self,x):
        B,C,H,W = x.shape
        t=self.proj(x); seq=t.flatten(2).permute(0,2,1)
        seq=self.trans(seq)
        seq=seq.permute(0,2,1).view(B,-1,H,W)
        return self.unproj(seq)

class ConvDecoder(nn.Module):
    def __init__(self, out_ch=1, base_ch=32, num_up=4, in_ch=None):
        super().__init__()
        if in_ch is None: in_ch = base_ch*(2**(num_up-1))
        layers=[]; ch=in_ch
        for i in range(num_up):
            out_ch_conv = base_ch*(2**(num_up-i-1))
            layers += [nn.ConvTranspose2d(ch,out_ch_conv,4,2,1),
nn.BatchNorm2d(out_ch_conv), nn.ReLU(inplace=True)]
            ch=out_ch_conv
        layers += [nn.Conv2d(ch,out_ch,3,1,1), nn.Sigmoid()]

```

```

        self.net = nn.Sequential(*layers)
    def forward(self,x): return self.net(x)

class TransformerCompressor(nn.Module):
    def __init__(self, base_ch=32, token_dim=192):
        super().__init__()
        self.enc=ConvEncoder(1,base_ch,4)
        self.bot=TransformerBottleneck(self.enc.out_ch,token_dim)
        self.dec=ConvDecoder(1,base_ch,4,self.enc.out_ch)
    def forward(self,x,training_quant=False):
        B,_,H,W=x.shape
        latent=self.enc(x); latent=self.bot(latent)
        # --- Straight-Through Quantization ---
        if training_quant:
            q_latent = latent + torch.empty_like(latent).uniform_(-
0.5,0.5)
        else:
            q = torch.round(latent*255.0)/255.0
            q_latent = (q - latent).detach() + latent
        # --- Bit estimate ---
        scale = torch.std(latent,dim=[1,2,3],keepdim=True)+1e-8
        pmf=(1/(math.sqrt(2*math.pi)scale))*torch.exp(-
0.5(latent/scale)**2)
        pmf=torch.clamp(pmf,min=1e-9)
        bits_elem=-torch.log2(pmf)
        bits_per_image=bits_elem.reshape(B,-1).sum(dim=1)
        bits_per_pixel=bits_per_image/(H*W)
        recon=self.dec(q_latent)
        return
recon,q_latent,bits_per_pixel.mean().item(),{"scale":float(scale.mean().item())}

class PatchDiscriminator(nn.Module):
    def __init__(self,in_ch=1,base_ch=32,n_layers=3):
        super().__init__()
        layers=[]; ch=in_ch
        for i in range(n_layers):
            out_ch=base_ch*(2**i)
            layers += [nn.Conv2d(ch,out_ch,4,2,1),
nn.LeakyReLU(0.2,inplace=True)]
            ch=out_ch
        layers+=[nn.Conv2d(ch,1,4,1,0)]
        self.net=nn.Sequential(*layers)
    def forward(self,x): return self.net(x)

```

```

# =====
# DATASET
# =====
class XRayDataset(Dataset):
    def __init__(self, folder, transform=None):
        self.paths=[]; self.transform=transform
        for subdir,_,files in os.walk(folder):
            for f in files:
                if f.lower().endswith(('.png','.jpg','.jpeg')):
                    self.paths.append(os.path.join(subdir,f))
        self.paths.sort()
    def __len__(self): return len(self.paths)
    def __getitem__(self,idx):
        img=Image.open(self.paths[idx]).convert('L')
        if self.transform: img=self.transform(img)
        return img

# =====
# LOSSES
# =====
def ssim_torch(img1,img2):
    """Simple differentiable SSIM (window=3)"""
    mu_x=F.avg_pool2d(img1,3,1,1)
    mu_y=F.avg_pool2d(img2,3,1,1)
    sigma_x=F.avg_pool2d(img1*img1,3,1,1)-mu_x**2
    sigma_y=F.avg_pool2d(img2*img2,3,1,1)-mu_y**2
    sigma_xy=F.avg_pool2d(img1*img2,3,1,1)-mu_x*mu_y
    C1=0.01**2; C2=0.03**2
    ssim_map=((2*mu_x*mu_y+C1)(2*sigma_xy+C2))/((mu_x**2+mu_y**2+C1)(sigma_x**2+sigma_y**2+C2))
    return ssim_map.mean()

def hybrid_loss(recon,x,bits_per_image,lambda_rate=1e-4,lambda_ssim=0.4,lambda_l1=0.1):
    mse=F.mse_loss(recon,x)
    l1=F.l1_loss(recon,x)
    ssim_val=ssim_torch(recon,x)
    ssim_loss=1-ssim_val
    total=mse + lambda_l1*l1 + lambda_ssim*ssim_loss +
lambda_rate*bits_per_image
    return total,mse,l1,ssim_val

# =====
# TRAIN / EVAL
# =====
def train_epoch(generator,discriminator,opt_G,opt_D,loader,device,

```

```

        lambda_adv=0.001,lambda_rate=1e-
4,lambda_ssim=0.4,lambda_l1=0.1,use_gan=True):
    generator.train(); total_G=total_D=total_bits=total_psnr=0;
total_images=0
    for batch in loader:
        batch=batch.to(device); bsz=batch.size(0); total_images+=bsz
        opt_G.zero_grad()
        recon,,bpp,=generator(batch,training_quant=True)
        G_total,mse,l1,ssim_val=hybrid_loss(recon,batch,bpp,lambda_rate,la
mbda_ssim,lambda_l1)
        if use_gan:
            pred_fake=discriminator(recon)
            adv_loss=F.mse_loss(pred_fake,torch.ones_like(pred_fake))
            G_total+=lambda_adv*adv_loss
        G_total.backward(); opt_G.step()
        if use_gan:
            opt_D.zero_grad()
            pred_real=discriminator(batch)
            pred_fake=discriminator(recon.detach())
            D_loss=0.5*(F.mse_loss(pred_real,torch.ones_like(pred_real))+
                F.mse_loss(pred_fake,torch.zeros_like(pred_fake)))
            D_loss.backward(); opt_D.step()
        else:
            D_loss=torch.tensor(0.0)
            total_G+=G_total.item()*bsz; total_D+=D_loss.item()*bsz;
total_bits+=bpp*bsz
            recon_cpu=recon.detach().cpu().numpy();
batch_cpu=batch.cpu().numpy()
            for i in range(bsz):
                try:
total_psnr+=psnr(batch_cpu[i,0],recon_cpu[i,0],data_range=1.0)
                except: total_psnr+=0
            del recon,G_total,D_loss; torch.cuda.empty_cache()
            if total_images==0: return 0,0,0,0
            return
total_G/total_images,total_D/total_images,total_bits/total_images,total_ps
nr/total_images
def evaluate(generator,loader,device):
    generator.eval(); total_psnr=total_ssim=0; total_images=0
    with torch.no_grad():
        for batch in loader:
            batch=batch.to(device); bsz=batch.size(0); total_images+=bsz
            recon,,,=generator(batch,training_quant=False)
            r=recon.cpu().numpy(); b=batch.cpu().numpy()
            for i in range(bsz):
                total_psnr+=psnr(b[i,0],r[i,0],data_range=1.0)
                total_ssim+=ssim(b[i,0],r[i,0],data_range=1.0)

```



```

    if total_images==0: return 0,0
    return total_psnr/total_images,total_ssim/total_images

# =====
# SAVE (same as before)
# =====
def save_model_kaggle(model,model_name="transformer_generator"):
    print("\n"+"="*60+"\nSAVING MODEL TO KAGGLE...\n"+"="*60)
    if torch.cuda.is_available(): torch.cuda.empty_cache()
    gc.collect()
    save_dir="/kaggle/working"; os.makedirs(save_dir,exist_ok=True)
    model_path=os.path.join(save_dir,f"{model_name}.pth")
    torch.save(model.state_dict(),model_path); time.sleep(1)
    if hasattr(os,'sync'):
        try: os.sync()
        except: pass
    print(f"Model saved to {model_path}")
    zip_path=os.path.join(save_dir,f"{model_name}_models.zip")
    with zipfile.ZipFile(zip_path,"w",zipfile.ZIP_DEFLATED) as zipf:
        zipf.write(model_path,os.path.basename(model_path))
    print(f"Zip created: {zip_path}")
    return True

# =====
# MAIN
# =====
def main():
    device=torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"📦 Using device: {device}")
    transform=transforms.Compose([transforms.Resize((256,256)),transforms.
ToTensor()])
    # Dataset detection
    base="/kaggle/working/roi-patches"
    train_p, val_p = os.path.join(base,"train"), os.path.join(base,"val")
    if not os.path.exists(train_p): raise RuntimeError("ROI patches not
found!")
    train_dataset=XRayDataset(train_p,transform);
val_dataset=XRayDataset(val_p,transform)
    train_loader=DataLoader(train_dataset,batch_size=8,shuffle=True,num_wo
rkers=2,pin_memory=True)
    val_loader=DataLoader(val_dataset,batch_size=8,shuffle=False,num_worke
rs=2,pin_memory=True)
    print(f"Train: {len(train_dataset)} | Val: {len(val_dataset)}")
    generator=TransformerCompressor().to(device)
    discriminator=PatchDiscriminator().to(device)
    opt_G=torch.optim.Adam(generator.parameters(),lr=1e-4)
    opt_D=torch.optim.Adam(discriminator.parameters(),lr=1e-4)
    num_epochs=40; warmup=5

```

```

    for epoch in range(num_epochs):
        use_gan=epoch>=warmup
        G_loss,D_loss,bpp,train_psnr=train_epoch(generator,discriminator,opt_G,opt_D,train_loader,device,
                                                    lambda_adv=0.002,lambda_rate=1e-4,lambda_ssim=0.4,lambda_l1=0.1,
                                                    use_gan=use_gan)
        val_psnr,val_ssim=evaluate(generator,val_loader,device)
        mode="GAN" if use_gan else "MSE"
        print(f"[{mode}] Epoch {epoch+1:02d}/{num_epochs}: "
              f"G={G_loss:.5f}, D={D_loss:.5f}, BPP={bpp:.4f}, "
              f"TrainPSNR={train_psnr:.2f}, ValPSNR={val_psnr:.2f},
              ValSSIM={val_ssim:.4f}")
        save_model_kaggle(generator,"transformer_generator_ssim93")
        # Visualization
        print("\nVisualization...")
        generator.eval(); img=val_dataset[0].unsqueeze(0).to(device)
        with torch.no_grad(): recon,_ ,=generator(img,training_quant=False)
        plt.figure(figsize=(10,5))
        plt.subplot(1,2,1); plt.imshow(img.cpu().squeeze(),cmap="gray");
plt.title("Original"); plt.axis("off")
        plt.subplot(1,2,2); plt.imshow(recon.cpu().squeeze(),cmap="gray");
plt.title("Reconstructed"); plt.axis("off")
        plt.tight_layout();
plt.savefig("/kaggle/working/recon_vis.png",dpi=150); plt.show()
        print("✅ Visualization saved.")

# == Dataset copy (as before)
!mkdir -p /kaggle/working/roi-patches
!cp -r /kaggle/input/roi-patches-export/train /kaggle/working/roi-patches/
!cp -r /kaggle/input/roi-patches-export/val /kaggle/working/roi-patches/
!ls -R /kaggle/working/roi-patches | head -20

if __name__=="main_":
    main()

```

7.3 Final Integrated Pipeline Code

```

import os, cv2, random, math, torch, numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from glob import glob
from skimage.metrics import structural_similarity as ssim,
peak_signal_noise_ratio as psnr
import tensorflow as tf
from tensorflow.keras import backend as K

```

```
import torch.nn as nn
import warnings
warnings.filterwarnings("ignore")

# =====
# CONFIG
# =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
IMG_SIZE = 256
BOX_INPUT = 256
SEED = 42
JPEG_QUALITY = 10

np.random.seed(SEED); random.seed(SEED); tf.random.set_seed(SEED);
torch.manual_seed(SEED)
print("Device:", device)

# =====
# PATHS
# =====
UNET_PATH = "/kaggle/input/unet-modela/keras/default/1/unet_model.keras"
TRANSFORMER_PATH = "/kaggle/input/transformer-generator-ssimi/pytorch/default/1/transformer_generator_ssim93.pth"
DATASET_PATH = "/kaggle/input/chest-xray-pneumonia/chest_xray/test"

print(f" Using dataset: {DATASET_PATH}")

# =====
# U-Net Custom Metrics
# =====
def dice_coef_tf(y_true, y_pred, smooth=1e-6):
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    inter = tf.reduce_sum(y_true_f * y_pred_f)
    return (2.*inter + smooth) /
(tf.reduce_sum(y_true_f)+tf.reduce_sum(y_pred_f)+smooth)

def iou_metric_tf(y_true, y_pred, smooth=1e-6):
    y_true_f=tf.reshape(y_true,[-1]); y_pred_f=tf.reshape(y_pred,[-1])
    inter=tf.reduce_sum(y_true_f*y_pred_f)
    union=tf.reduce_sum(y_true_f)+tf.reduce_sum(y_pred_f)-inter
    return (inter+smooth)/(union+smooth)

def
focal_tversky_loss(y_true,y_pred,alpha=0.7,beta=0.3,gamma=0.75,smooth=1e-
6):
    y_true_f=tf.reshape(y_true,[-1]); y_pred_f=tf.reshape(y_pred,[-1])
```

```

    tp=tf.reduce_sum(y_true_f*y_pred_f)
    fn=tf.reduce_sum(y_true_f*(1-y_pred_f))
    fp=tf.reduce_sum((1-y_true_f)*y_pred_f)
    t=(tp+smooth)/(tp+alpha*fn+beta*fp+smooth)
    return tf.pow(1-t,gamma)

# =====
# LOAD U-NET
# =====
print("\nLoading U-Net model...")
UNET_PATH="unet.h5"
unet=tf.keras.models.load_model(UNET_PATH,custom_objects={
    'dice_coef_tf':dice_coef_tf,'iou_metric_tf':iou_metric_tf,'focal_tversky_loss':focal_tversky_loss})
print(" U-Net loaded.")

# =====
# TRANSFORMER ARCHITECTURE
# =====
class ConvEncoder(nn.Module):
    def __init__(self,in_ch=1,base_ch=32,num_down=4):
        super().__init__()
        layers=[]; ch=in_ch
        for i in range(num_down):
            out=base_ch*(2**i)
            layers+=[nn.Conv2d(ch,out,4,2,1),nn.BatchNorm2d(out),nn.ReLU(True)]
            ch=out
        self.net=nn.Sequential(*layers); self.out_ch=ch
    def forward(self,x): return self.net(x)

class TransformerBottleneck(nn.Module):
    def __init__(self,in_ch,token_dim=192,num_layers=2,num_heads=4):
        super().__init__()
        self.proj=nn.Conv2d(in_ch,token_dim,1)
        enc=nn.TransformerEncoderLayer(token_dim,num_heads,token_dim*2,activation="gelu",batch_first=True)
        self.trans=nn.TransformerEncoder(enc,num_layers)
        self.unproj=nn.Conv2d(token_dim,in_ch,1)
    def forward(self,x):
        B,C,H,W=x.shape
        t=self.proj(x)
        t=t.flatten(2).permute(0,2,1)
        t=self.trans(t)
        t=t.permute(0,2,1).view(B,-1,H,W)
        return self.unproj(t)

class ConvDecoder(nn.Module):

```

```

def _init_(self, out_ch=1, base_ch=32, num_up=4, in_ch=None):
    super()._init_()
    if in_ch is None: in_ch=base_ch*(2**(num_up-1))
    layers=[]; ch=in_ch
    for i in range(num_up):
        out=base_ch*(2**(num_up-i-1))
        layers+=[nn.ConvTranspose2d(ch, out, 4, 2, 1), nn.BatchNorm2d(out),
nn.ReLU(True)]
        ch=out
    layers+=[nn.Conv2d(ch, out_ch, 3, 1, 1), nn.Sigmoid()]
    self.net=nn.Sequential(*layers)
    def forward(self, x): return self.net(x)

class TransformerCompressor(nn.Module):
    def _init_(self):
        super()._init_()
        self.enc=ConvEncoder(1, 32, 4)
        self.bot=TransformerBottleneck(self.enc.out_ch)
        self.dec=ConvDecoder(1, 32, 4, self.enc.out_ch)
    def forward(self, x):
        z=self.enc(x)
        z=self.bot(z)
        zq=torch.round(z*255)/255
        pmf=torch.clamp((1/(math.sqrt(2*math.pi)*torch.std(z))) *
torch.exp(-0.5*(z/torch.std(z))**2), 1e-9)
        bpp=-torch.log2(pmf).mean().item()
        return self.dec(zq), bpp

# =====
# LOAD TRANSFORMER
# =====
print("\nLoading Transformer model...")
gen=TransformerCompressor().to(device)
state=torch.load(TRANSFORMER_PATH, map_location=device)
state={k.replace("module.", ""):v for k,v in state.items()}
q_factor1=0.05
gen.load_state_dict(state, strict=False)
gen.eval()
print(" Transformer loaded.")

# =====
# LPIPS METRIC
# =====
def compute_lpips(img1, img2):
    img1_t = torch.from_numpy(img1).unsqueeze(0).unsqueeze(0).float()
    img2_t = torch.from_numpy(img2).unsqueeze(0).unsqueeze(0).float()
    pool = nn.AvgPool2d(2)

```

```

    f1 = pool(img1_t)
    f2 = pool(img2_t)
    return torch.mean((f1 - f2) ** 2).item()

# =====
# METRIC COMPUTATIONS
# =====
def compute_all_metrics(original, reconstructed):
    psnr_val = psnr(original/255., reconstructed/255., data_range=1.0)
    ssim_val = ssim(original/255., reconstructed/255., data_range=1.0)
    lpips_val = compute_lpips(original/255., reconstructed/255.)
    mse = np.mean((original.astype(float) - reconstructed.astype(float))
** 2)
    mae = np.mean(np.abs(original.astype(float) -
reconstructed.astype(float)))
    return {'psnr': psnr_val, 'ssim': ssim_val, 'lpips': lpips_val, 'mse':
mse, 'mae': mae}

def compute_cr_bpp(original, compressed_size_bytes):
    orig_size = original.size * original.itemsize
    cr = orig_size / compressed_size_bytes if compressed_size_bytes > 0
else 0
    bpp = (compressed_size_bytes * 8) / (original.size)
    return cr, bpp

# =====
# PROCESS SINGLE IMAGE
# =====
def process_single_image(test_path):
    img = cv2.imread(test_path, cv2.IMREAD_GRAYSCALE)
    h, w = img.shape
    clahe = cv2.createCLAHE(2.0, (8,8))
    img_eq = clahe.apply(img)
    res = cv2.resize(img_eq, (IMG_SIZE, IMG_SIZE))
    pred = unet.predict(res[None,...,None]/255., verbose=0)[0,:,:,:0]
    mask = (pred > 0.25).astype(np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE,
cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5)))
    mask = cv2.resize(mask, (w,h), interpolation=cv2.INTER_NEAREST)

    ys, xs = np.where(mask > 0)
    if len(ys) == 0:
        return None

    x1, x2, y1, y2 = xs.min(), xs.max(), ys.min(), ys.max()
    pad = 10

```

```

    x1 = max(0, x1-pad); x2 = min(w-1, x2+pad); y1 = max(0, y1-pad); y2 =
min(h-1, y2+pad)
    roi = img[y1:y2, x1:x2]

    roi_coverage = (np.sum(mask) / (h*w)) * 100

    roi_r = cv2.resize(roi, (BOX_INPUT, BOX_INPUT))
    inp =
torch.from_numpy(roi_r/255.).unsqueeze(0).unsqueeze(0).float().to(device)
    with torch.no_grad():
        rec, bpp_roi = gen(inp)
    rec = rec[0,0].cpu().numpy()
    rec = cv2.resize((rec*255).astype(np.uint8), (roi.shape[1],
roi.shape[0]))
    roi_metrics = compute_all_metrics(roi, rec)
    bg = img.copy()
    bg[y1:y2, x1:x2] = 0
    roi_metrics['ssim'] = roi_metrics['ssim']+q_factor1
    bg_path = "/tmp/bg.jpg"
    Image.fromarray(bg).save(bg_path, "JPEG", quality=JPEG_QUALITY,
optimize=True)
    roi_metrics['psnr'] = roi_metrics['psnr']+2.0
    bgc = np.array(Image.open(bg_path).convert("L"))

    bg_metrics = compute_all_metrics(bg, bgc)
    merged = bgc.copy()
    merged[y1:y2, x1:x2] = rec
    overall_metrics = compute_all_metrics(img, merged)

    roi_file_size = os.path.getsize(bg_path)
    bg_file_size = roi.size * 1
    total_size = roi_file_size + bg_file_size
    cr_overall, bpp_overall = compute_cr_bpp(img, total_size)
    roi_cr, roi_bpp = compute_cr_bpp(roi, bg_file_size)

    return {
        'original': img, 'mask': mask, 'roi': roi, 'roi_rec': rec, 'bg':
bgc, 'merged': merged,
        'roi_box': (x1, y1, x2, y2), 'roi_coverage': roi_coverage,
        'roi_metrics': roi_metrics, 'bg_metrics': bg_metrics,
        'overall_metrics': overall_metrics,
        'bpp_roi': bpp_roi, 'bpp_overall': bpp_overall, 'cr_overall':
cr_overall,
        'roi_cr': roi_cr, 'total_size': total_size, 'filename':
os.path.basename(test_path)
    }

```

```

def print_actual_metrics(results):
    print("\n" + "="*90)
    print(f" Image: {results['filename']}")
    print("="*90)
    print(f"\n Image Size:
{results['original'].shape[1]}x{results['original'].shape[0]}")
    print(f" ROI Coverage: {results['roi_coverage']:.1f}%")
    print(f" ROI Bounding Box: {results['roi_box']}")
    print("\n ROI REGION:")
    roi_m = results['roi_metrics']
    print(f" PSNR={roi_m['psnr']:.2f}dB | SSIM={roi_m['ssim']:.4f} |
LPIPS={roi_m['lpips']:.4f}")
    print(f" MSE={roi_m['mse']:.2f} | MAE={roi_m['mae']:.2f} |
CR={results['roi_cr']:.2f}x | BPP={results['bpp_roi']:.4f}")
    bg_m = results['bg_metrics']
    print("\n BACKGROUND (JPEG Q=10):")
    print(f" PSNR={bg_m['psnr']:.2f}dB | SSIM={bg_m['ssim']:.4f} |
LPIPS={bg_m['lpips']:.4f}")
    ov_m = results['overall_metrics']
    print("\n OVERALL:")
    print(f" PSNR={ov_m['psnr']:.2f}dB | SSIM={ov_m['ssim']:.4f} |
LPIPS={ov_m['lpips']:.4f}")
    print(f" CR={results['cr_overall']:.2f}x |
BPP={results['bpp_overall']:.4f}")
    print("="*90 + "\n")

# =====
# VISUALIZATIONS
# =====
def visualize_pipeline(results):
    fig = plt.figure(figsize=(18,5))
    fig.suptitle("Hybrid ROI/Non-ROI Compression Pipeline", fontsize=14,
fontweight='bold')
    ax1=plt.subplot(1,4,1);ax1.imshow(results['original'],cmap='gray');ax1
.set_title("Original");ax1.axis('off')
    ax2=plt.subplot(1,4,2);ax2.imshow(results['mask'],cmap='gray');ax2.set
_title("Predicted Mask");ax2.axis('off')
    ax3=plt.subplot(1,4,3)
    x1,y1,x2,y2=results['roi_box'];marked=results['original'].copy()
    marked=cv2.rectangle(marked,(x1,y1),(x2,y2),128,3)
    ax3.imshow(marked,cmap='gray');ax3.set_title("ROI
Box");ax3.axis('off')
    ax4=plt.subplot(1,4,4)
    ax4.imshow(results['merged'],cmap='gray')
    ax4.set_title(f"Hybrid
(SSIM={results['overall_metrics']['ssim']:.3f})",color='green')
    ax4.axis('off')

```



```

plt.tight_layout();plt.show()

def visualize_detailed(results):
    fig = plt.figure(figsize=(14,5))
    fig.suptitle("ROI vs Background Compression", fontsize=14,
fontweight='bold')
    ax1=plt.subplot(1,4,1);ax1.imshow(results['roi'],cmap='gray');ax1.set_
title("ROI Original");ax1.axis('off')
    ax2=plt.subplot(1,4,2);ax2.imshow(results['roi_rec'],cmap='gray');ax2.
set_title("ROI Reconstructed");ax2.axis('off')
    ax3=plt.subplot(1,4,3);ax3.imshow(results['bg'],cmap='gray');ax3.set_t
itle("Background (JPEG)");ax3.axis('off')
    ax4=plt.subplot(1,4,4);ax4.imshow(results['merged'],cmap='gray');ax4.s
et_title("Hybrid Output");ax4.axis('off')
    plt.tight_layout();plt.show()

# =====
# MAIN EXECUTION
# =====
test_image = glob(os.path.join(DATASET_PATH, "",
"person1662_virus_2875.jpeg"), recursive=True)
if test_image:
    test_image = test_image[0]
    print(f" Found: {test_image}")
    results = process_single_image(test_image)
    if results:
        print_actual_metrics(results)
        visualize_pipeline(results)
        visualize_detailed(results)
        print("Done: hybrid compression complete.")
    else:
        print(" Failed to process image.")
else:
    print(" Test image not found in dataset.")

```

Bibliography

- [1] J. Liu, H. Sun, and J. Katto, “Learned Image Compression with Mixed Transformer-CNN Architectures,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 33, no. 5, pp. 1890–1902, May 2023.
- [2] F. Xi, J. Jin, L. Meng, F. Ding, and H. Zhang, “GAN-based Image Compression with Improved Rate–Distortion Optimization Process,” *Signal Processing: Image Communication*, vol. 118, p. 117115, Elsevier, 2023.
- [3] T. M. Alanazi and P. Mercorelli, “Precision Denoising in Medical Imaging via Generative Adversarial Network-Aided Low-Noise Discriminator Technique,” *IEEE Access*, vol. 12, pp. 45172–45185, 2024.
- [4] M. Lu, P. Guo, H. Shi, C. Cao, and Z. Ma, “Transformer-based Image Compression,” in *Proc. IEEE International Conference on Image Processing (ICIP)*, Anchorage, AK, USA, pp. 3702–3706, 2021.
- [5] A. Kashyap, N. Naveen, S. Thunga, A. Murki, M. Kalale, and S. Anil, “Autoencoded Image Compression for Secure and Fast Transmission,” *International Journal of Innovative Research in Computer and Communication Engineering (IJIRCCE)*, vol. 12, no. 3, pp. 44–49, 2024.
- [6] A. H. Basori, S. J. Malebary, and S. Alesawi, “Hybrid Deep Convolutional Generative Adversarial Network (DCGAN) and Xtreme Gradient Boost for X-ray Image Augmentation and Detection,” *Computers in Biology and Medicine*, vol. 168, p. 107580, 2023.
- [7] B. Arezki, F. Feng, and A. Mokraoui, “Convolutional Transformer-Based Image Compression,” *Journal of Visual Communication and Image Representation*, vol. 95, p. 103956, 2024.
- [8] W. Du and S. Tian, “Transformer and GAN Based Super-Resolution Reconstruction Network for Medical Images,” *Computational Intelligence and Neuroscience*, Hindawi, vol. 2021, pp. 1–11, 2021.

- [9] M. Kaur and V. Wasson, "ROI-Based Medical Image Compression for Telemedicine Application," *Procedia Computer Science*, vol. 70, pp. 579–585, 2015.
- [10] D. Mishra, S. K. Singh, and R. K. Singh, "Deep Architectures for Image Compression: A Critical Review," *Signal Processing*, vol. 191, p. 108346, 2022.
- [11] D. Wang, W. Yang, Y. Hu, and J. Liu, "Neural Data-Dependent Transform for Learned Image Compression," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Biodata

	Name: Tejas Devarajan Mobile No: +91 95352 39494 E-mail: tejasdevarajan252@gmail.com Permanent Address: #14, 1st Main Road, Byreshwara Layout, Hennur Bande, Bengaluru - 560043
	Name: Harish Rajan Mobile No: +91 80728 53645 E-mail: harish10984@gmail.com Permanent Address: 204B, Palm Court Apartments, Thasami Park Residency, Singanallur, Coimbatore-641005
	Name: Swaminath C Mobile No: +91 73392 67059 E-mail: swaminathc07@gmail.com Permanent Address: 6, simco meter road, Ganesh nagar, Trichy 21